



Northern University

Of Business and Technology Khulna

Course Title: Mobile Computing Lab

Course Code: CSE 4204

AI Integration Report

Submitted by:

Name: Nazmus Sakib
Section: 8B
ID: 11220320888
Department: Computer Science and
Engineering

Submitted to:

Name: Md. Riaz Mahmud
Designation: Assistant Professor
Department: Computer Science and
Engineering

Date of Submission: 13.08.2026

Table of Contents

Table of Contents	1
1. Project Overview	2
2. Team Information	2
3. Why AI Integration Is Required	2
4. Selected AI Platform	3
5. AI Model Used.....	3
6. AI Features Implemented.....	3
7. AI Workflow	4
8. Prompt Engineering	5
8.1 HR Agent - CV Screening.....	6
8.2 Finance Agent - Anomaly Detection.....	6
8.3 Support Agent - Grounded Reply Drafting (RAG).....	6
9. AI Response Handling Strategy	7
10. AI Output Validation	7
11. Current Development Progress	8
12. Current Limitations	8
13. Future Improvements	8
14. GitHub Repository Link	9

Enterprise Nexus — Multi-Agent AI Business Automation OS with Security Intelligence

1. Project Overview

Enterprise Nexus — Multi-Agent AI Business Automation OS with Security Intelligence.

Enterprise Nexus unifies HR, finance, customer support, analytics and executive decision-making under one AI-driven operating system, deploying five specialized agents that reason over an organization's own business data instead of generic, ungrounded chat.

2. Team Information

Field	Detail
Course	CSE 4204 - Mobile Computing Lab
Team ID	CSE4204-8B-T04
Project	Enterprise Nexus — Multi-Agent AI Business Automation OS with Security Intelligence
Supervisor	Md. Riaz Mahmud (Assistant Professor)
Deliverable	Week 08 - AI Integration Report

Name	Student ID	Role
Nazmus Sakib	11220320888	Team Leader & AI Integration Lead
Shoeb Shariar Mashuk	11220320878	Backend Developer
Most Sumiya Sanjida	11220320874	Frontend Developer - 1
Sabrina Ibrahim	11220320895	Database Manager & Frontend Developer - 2

3. Why AI Integration Is Required

Enterprise Nexus targets small and medium businesses that cannot afford dedicated HR analysts, finance auditors or support triage staff. Manually screening every CV, auditing every expense line and triaging every support ticket does not scale as an organization grows. AI is not used here as a demonstration feature - it is the mechanism that makes the platform's core promise possible: automated, explainable, data-grounded decision support across five business domains, available the moment a record is created rather than after a human reviewer gets to it.

Without AI, the platform would be a conventional CRUD dashboard. With it, every CV is scored and ranked, every expense is checked for anomalies, every ticket is triaged and answered with cited knowledge and cross-domain executive summaries are generated automatically - the measurable value the assignment brief asks for.

4. Selected AI Platform

Google Gemini was selected as the AI platform, accessed through the official @google/generative-ai SDK and orchestrated with LangChain.js (@langchain/google-genai) for tool-calling agent behavior.

Criterion	Reason Gemini Was Chosen
Structured output	Native JSON mode with a response schema returns strictly typed JSON - no markdown fences to strip, no brittle regex parsing.
Cost & quota	Gemini 2.5 Flash offers a generous free tier (~60 requests/min), sufficient for iterative development and demo load.
Multimodal input	Accepts PDF/DOCX-extracted text and image content for CV parsing and invoice OCR without a second vendor.
Embeddings	text-embedding-004 (768-dim) integrates with the same SDK and billing, keeping the whole AI layer on one platform.
Ecosystem fit	First-class LangChain.js support (ChatGoogleGenerativeAI) enabled the BaseAgent tool-calling loop directly.

5. AI Model Used

Purpose	Model	Notes
Reasoning / scoring / summarization	gemini-2.5-flash	Configurable via GEMINI_MODEL env var; JSON mode + response schema enforced on every call.
Embeddings (RAG)	text-embedding-004	768-dimensional vectors stored in Supabase pgvector.
Agent orchestration	ChatGoogleGenerativeAI (LangChain.js)	Wraps the same Gemini model for tool-calling agents.

6. AI Features Implemented

- **HR Agent : Intelligent CV Screening** - CVs parsed from PDF/DOCX, scored 0–100 by Gemini in native JSON mode, with confidence, a shortlist/review/reject recommendation, a per-dimension score breakdown and a narrative summary.
- **Finance Agent : Anomaly Detection** - Every expense record is analyzed and assigned a severity (normal / medium / high / critical) with an explanatory note; invoices and spreadsheets are parsed automatically.

- **Support Agent : Context Aware Support** - Incoming tickets are analyzed for intent, urgency and sentiment; replies are drafted using Retrieval-Augmented Generation (RAG) grounded in the knowledge base, with automatic escalation for urgent cases.
- **Analytics Agent** - Computes cross-module KPI snapshots (HR shortlist rate, finance spend/anomaly rate, support resolution/escalation rate) and generates full AI performance reports.
- **Executive Agent : Cross-Domain Orchestration** - A true multi-agent orchestrator - its tools are the four specialist agents. It delegates cross-domain questions and produces synthesized daily executive briefings, exportable to PDF.
- **Semantic Search / RAG:** Documents and business records are chunked, embedded and stored in pgvector; the match documents RPC returns the top-5 most relevant chunks by cosine similarity for grounded, citable answers.

7. AI Workflow

Enterprise Nexus follows the standard User Input → Frontend → Backend API → AI Service → AI Response → Frontend Display pipeline recommended in the course handbook, extended with an agentic tool-calling loop, output validation and a graceful-degradation branch so the application keeps working even when Gemini is temporarily unavailable:

- The user submits a task, query, or file (CV, invoice, ticket) through the Next.js frontend.
- The request reaches the Express backend, which authenticates the user and routes it to the relevant agent controller.
- The target agent (built on BaseAgent) runs a bounded tool-calling loop (max. 5 iterations), calling domain tools such as query candidates or retrieve knowledge to fetch the user's own real data - never inventing it.
- The assembled context is sent to Gemini in native JSON mode with a response schema, so the model returns strictly structured output.
- The response is validated against a Joi/Zod schema. If valid, it is persisted and marked Ai status: success; if the AI is unavailable or output fails validation, the record is stored as pending/failed and can be retried - no fabricated data is ever shown.
- The frontend displays the AI-generated result (score, recommendation, summary, or briefing) on the relevant dashboard, with real-time updates via Supabase Realtime.

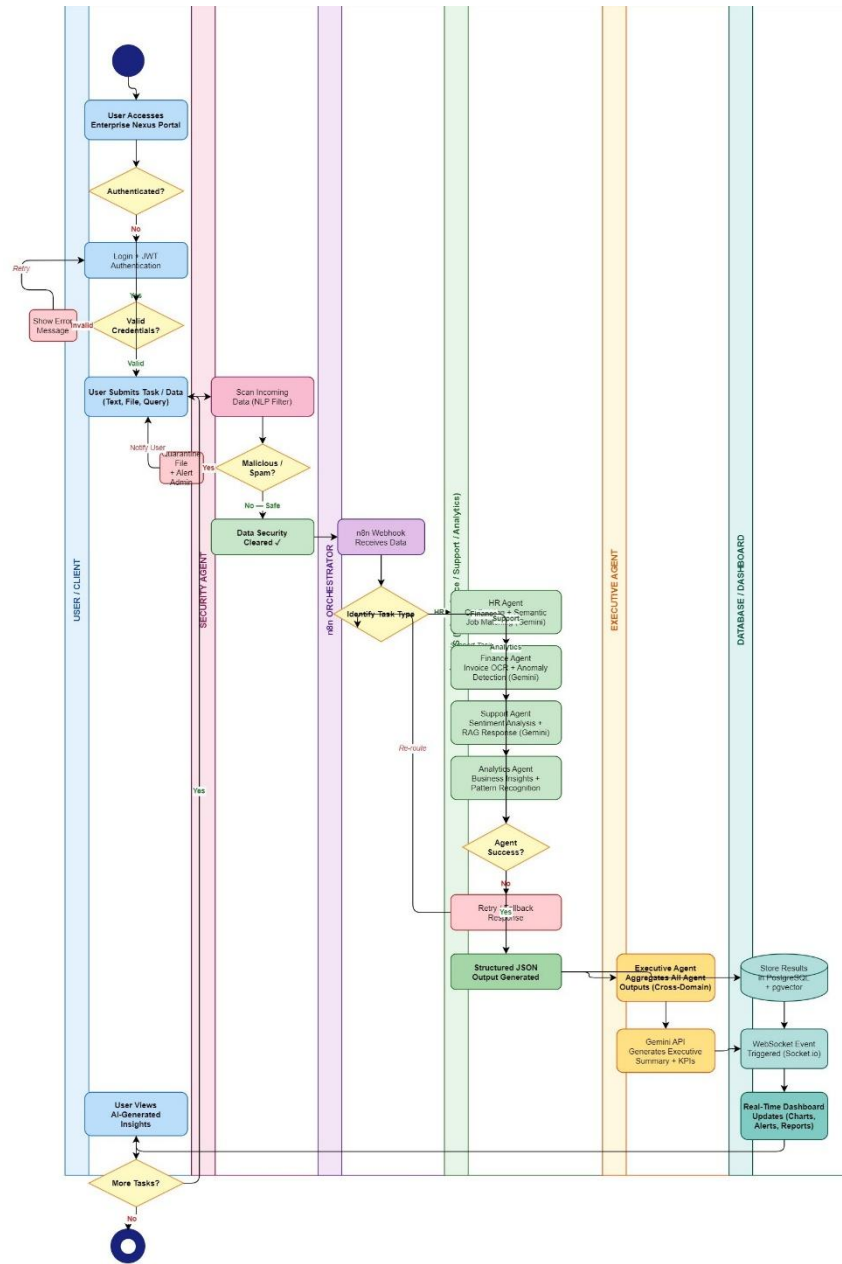


Figure 1 - AI Multi-Agent Workflow Activity Diagram

Full-resolution-diagram: <documentation/diagram/ai-workflow/00-ai-multi-agent-workflow-activity-diagram.jpg>

8. Prompt Engineering Example

Every agent's system prompt (a) fixes its role and domain, (b) explicitly forbids inventing data - "use your tools to fetch real data before answering; never invent candidates or scores" - (c) enforces per-user data isolation and (d) requires a strict JSON output shape via Gemini's native response-schema mode. Multiple prompt variations were tested for each agent during development - using [OpenAI Playground](#) and [Google AI Studio](#) to iterate on phrasing and compare output quality

before finalizing - to reduce hallucination and tighten output structure before settling on the versions below.

8.1 HR Agent - CV Screening

System Prompt	<i>You are the HR Agent for Enterprise Nexus. Using only the extracted CV text and job requirements provided via tools, score the candidate objectively. Never invent experience, skills, or qualifications not present in the source text. Respond with ONLY valid JSON matching the given schema.</i>
User Prompt	<i>Screen this candidate's CV against the Senior Backend Engineer role and return an AI score, confidence, recommendation, score breakdown and narrative summary.</i>
Expected Output	<pre>{ "ai_score": 82, "confidence": "high", "recommendation": "shortlist", "score_breakdown": { "skills_match": 85, "experience": 80, "education": 78 }, "narrative_summary": "Strong Node.js/PostgreSQL background with 4 years of relevant experience; recommended for a technical interview." }</pre>

8.2 Finance Agent - Anomaly Detection

System Prompt	<i>You are the Finance Agent for Enterprise Nexus. Analyze the given expense record strictly against the user's historical spending pattern supplied via tools. Assign a severity of normal, medium, high, or critical and justify it in one sentence. Never fabricate comparison figures.</i>
User Prompt	<i>Analyze this expense: "Office Supplies - BDT 185,000 - submitted by Marketing Dept" against this department's 6-month average of BDT 22,000.</i>
Expected Output	<pre>{ "severity": "critical", "note": "This expense is roughly 8.4x the department's 6-month average for this category and warrants manual review before approval." }</pre>

8.3 Support Agent - Grounded Reply Drafting (RAG)

System Prompt	<i>You are the Support Agent for Enterprise Nexus. Classify the ticket's intent, urgency and sentiment, then draft a reply using ONLY the knowledge-base chunks returned by the retrieve_knowledge tool. Cite the source title for every claim. If no relevant chunk is found, say so instead of guessing.</i>
User Prompt	<i>Ticket: "My invoice export button does nothing when I click it - I need this for a client meeting in an hour."</i>
Expected Output	<pre>{ "intent": "bug_report", "urgency": "high", "sentiment": "frustrated", "escalate": true, "draft_reply": "Sorry for the trouble - this matches a known issue with large exports (see 'Invoice Export Troubleshooting'). Try</pre>

	exporting a smaller date range while we escalate this for you.", "sources": ["Invoice Export Troubleshooting"] }
--	--

9. AI Response Handling Strategy

All Gemini calls are routed through a single hardened client (backend/src/ai/client.js) so every failure mode is handled in one place rather than scattered across controllers:

Scenario	Handling Strategy
Successful response	Parsed as strict JSON, validated against a Joi/Zod schema, then persisted with ai_status: success.
Invalid / malformed response	Raised as a typed AiOutputValidationError; not retried (retrying a bad prompt wastes quota) - record saved as failed for manual retry.
Empty response	Fails JSON parsing → treated as invalid output → same failed path, never displayed as a blank success.
API errors (4xx/5xx)	5xx and 429 classified as transient and retried; other 4xx surfaced immediately as AiUnavailableError.
Network failures	ECONNRESET / ETIMEDOUT / fetch failed pattern-matched as transient and retried with backoff.
Rate limits (429)	Detected via status code and message pattern; retried with exponential backoff + jitter.
Timeouts	Each attempt is wrapped in a 30-second timeout (AI_TIMEOUT_MS); a timeout is treated as transient and retried.

Up to two retries (three attempts total) are performed for transient failures, with exponential backoff plus random jitter to avoid thundering-herd retries. When every attempt is exhausted, the error surfaces as a typed AiUnavailableError with a reason code (timeout, rate_limit, api_error, invalid_output, not_configured). Controllers catch this and degrade gracefully - the record is stored with ai_status: pending or failed and a retry endpoint, so the rest of the application keeps functioning even while Gemini is down. This behaviour is enforced by a dedicated automated test suite (ai.nofakedata tests) that specifically verifies no fabricated data is ever returned to the user.

10. AI Output Validation

- Gemini's native response Schema mode plus a server-side Joi/Zod schema reject any response missing required fields before it reaches the database.

- A parse failure (non-JSON or empty text) is caught explicitly and routed to the failed path - never rendered to the user as a result.
- Score fields are range-checked (e.g., Ai score clamped to 0–100); free-text fields are trimmed and stripped of stray markdown/code-fence artifacts.
- Idempotency keys on write paths and status checks before re-analysis prevents the same record from generating duplicate AI outputs.
- On any validation failure, the UI shows a clear, actionable status ("Analysis pending - retry") instead of a silent blank state or a fabricated placeholder score.

11. Current Development Progress

As of this Week 08 milestone, the AI layer is functionally complete and integrated end-to-end:

- All five agents (HR, Finance, Support, Analytics, Executive) are implemented, connected to the backend and rendering AI-generated results on their respective frontend dashboards.
- The RAG pipeline (chunking → embeddings → pgvector → match_documents) is live for both the Support and Executive agents.
- The hardened Gemini client, typed error hierarchy and no-fake-data guarantee are implemented and covered by an automated test suite (70/70 backend tests passing at last verification).
- Five n8n workflows automate recurring AI-driven processes: HR shortlist pipeline, finance anomaly alerts, support auto-escalation, a daily executive briefing and a weekly analytics KPI job.
- An independent engineering audit (2026-07-19) re-verified linting, type-checking, the full test suite and a production build, alongside live runtime probes for auth, rate-limiting and error handling.

12. Current Limitations

- The system depends entirely on Gemini's availability and quota; under sustained outages, AI-dependent features fall back to a pending state rather than functioning offline.
- Frontend automated test coverage is lighter than backend coverage; a fuller Vitest/RTL pass over the AI-facing screens is a natural next step.
- Row-level data isolation is enforced at the application layer (scopedQuery) rather than at the database layer (Supabase RLS with the service-role key), which is documented and mitigated but remains a manual-review-dependent safeguard for new controllers.
- The briefing cache used by the Executive agent is an in-memory Map, so it resets on redeploy and is not shared across multiple backend instances.

13. Future Improvements

- Move per-user data isolation from application-layer scoped queries to database-level Row Level Security policies in Supabase.

- Add a persistent, shared cache (e.g., Redis) for executive briefings so results survive redeploys and scale across instances.
- Expand automated frontend tests to cover the AI interaction flows (CV upload → score display, ticket triage → reply draft).
- Introduce a lightweight evaluation harness that replays historical CVs/tickets/expenses through updated prompts to measure regression before deploying prompt changes.
- Add streaming responses for the Executive Agent chat interface to reduce perceived latency on longer cross-domain queries.

14. GitHub Repository Link

The complete, actively maintained source code, documentation and commit history for Enterprise Nexus are available at:

<https://github.com/Nazmussakib247/CSE4204-8B-T04-Enterprise-Nexus-Multi-Agent-AI-Business-Automation-OS-with-Security-Intelligence>

AI-related source code is organized under **backend/src/ai** (hardened Gemini client, embeddings pipeline, typed errors) and **backend/src/agents** (five specialist agents, shared tools, conversation memory), as described in the README's AI Integration section.